

Stable Diffusion

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 11, 2026

1. Stable Diffusion overview: text-to-image foundations and key components
2. Motivation and quick overview of the diffusion process
3. Attention-U-Net backbone: CNN/U-Net, time embedding, and cross-attention
4. Text conditioning: word vectors, transformers, and CLIP
5. Latent Diffusion Models: VAE compression, cross-attention, and the Stable Diffusion pipeline
6. Guidance and sampling: classifier-free guidance, negative prompts, samplers, and distillation
7. Customization: Textual Inversion, DreamBooth, LoRA, and ControlNet
8. Applications and lineage: Stable Video Diffusion, GLIDE, DALL·E 2, Imagen, SD 2.0, SDXL, SD 3 / 3.5, Nano Banana 2
9. Summary, limitations, and references

What is Stable Diffusion?

- ▶ A state-of-the-art text-to-image generative model.
- ▶ Released by Stability AI and collaborators (2022).
- ▶ Open-source, enabling wide adoption and customization.



Advantages

- ▶ High-quality, diverse image generation.
- ▶ Efficient training and inference due to latent space operations.
- ▶ Open-source and extensible for research and applications.

What do we need to make a good generative model?

1. Method of learning to generate new images
2. Way to link text and images
3. Way to compress images
4. Way to add in good inductive biases

Forward/reverse diffusion

Text-image representation model

Autoencoder

U-net architecture + 'attention'

Making a 'good' generative model is about making all these parts work together well!

Generative Modeling

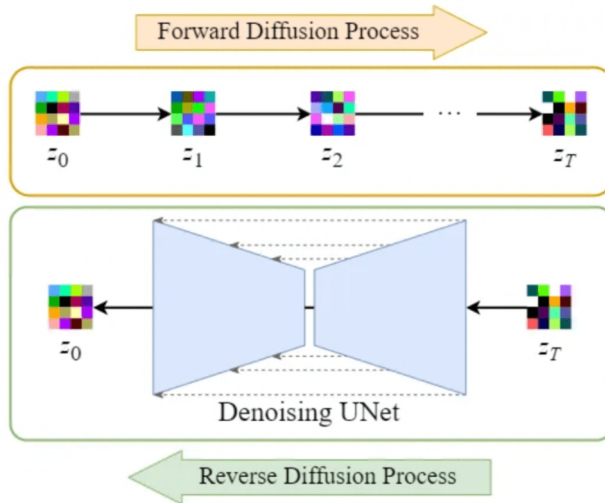
- ▶ Early successes: GANs and VAEs enabled impressive image synthesis and data generation.
- ▶ Drawbacks:
 - GANs: Training instability, mode collapse, and sensitivity to hyperparameters.
 - VAEs: Blurry outputs and limited expressiveness.
- ▶ Key Challenge: Achieving both high sample quality and stable, reliable training.

Where do we go from here?

- ▶ Need for models that are robust, interpretable, and capable of generating diverse, high-fidelity samples.

Enter Diffusion Models:

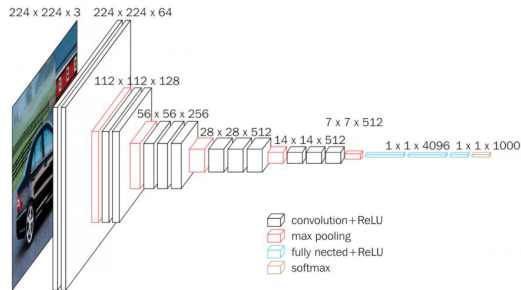
- ▶ Inspired by thermodynamics, diffusion models gradually transform noise into data.
- ▶ **Highlights:**
 - Simple training objective
 - Stable optimization
 - State-of-the-art sample quality
 - Probabilistic and interpretable framework



- ▶ **Forward:** Clean $\rightarrow$ Noise (additive Gaussian steps)
- ▶ **Reverse:** Learn **U-Net** to remove noise step by step
- ▶ **Training:** Model predicts the noise added
- ▶ **Sampling:** Roll back from pure noise to a data sample

Stable Diffusion Core Backbone: **Attention-U-Net Architecture**

- ▶ CNN parametrizes a function over images.
- ▶ **Motivation:**
 - Features are translationally invariant.
 - Extract features at different scales / abstraction levels.
- ▶ **Key modules:**
 - Convolution
 - Downsampling (max-pool)
- ▶ Features of larger scale (larger receptive field)
⇒ higher abstraction level.



Binxu Wang, [Stable Diffusion: A Tutorial \(Harvard ML from Scratch\)](#).

- ▶ An inverted CNN (generator) can generate images.
- ▶ CNN + inverted CNN can model an image-to-image function.
- ▶ **Down sampling:** convolution.
- ▶ **Up sampling:** transposed convolution.

أكاديمية كاوست
KAUST ACADEMY

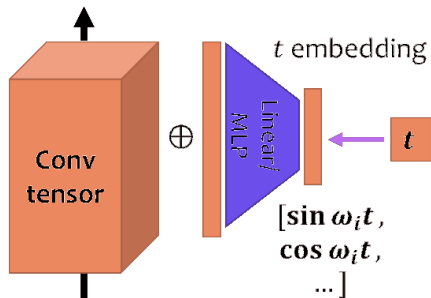


The score function is time-dependent.

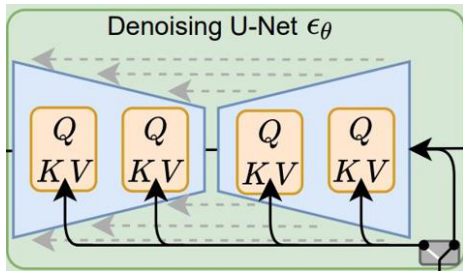
$$\text{Target: } s(x, t) = \nabla_x \log p(x, t)$$

Add time dependency

- ▶ Assume time dependency is spatially homogeneous.
- ▶ Add one scalar value per channel $f(t)$.
- ▶ Parametrize $f(t)$ by MLP/linear of Fourier basis.
 - Fourier features: $[\sin \omega_i t, \cos \omega_i t, \dots]$



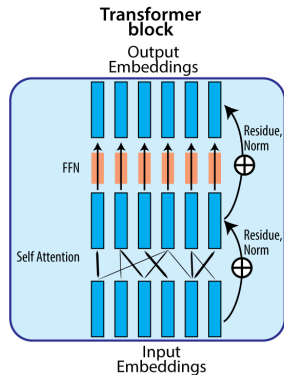
Binxu Wang, [Stable Diffusion: A Tutorial \(Harvard ML from Scratch\)](#).



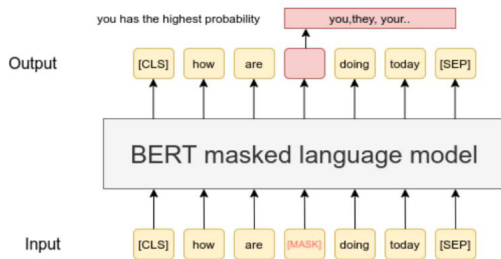
- ▶ Input convolution: Projects 4-channel input $\rightarrow$ 320 channels using a 2D convolution.
- ▶ Time embedding: inject time information into the network.
- ▶ Downsampling: sometimes interleaved with cross-attention.
- ▶ Mid block: applies additional residual and (cross-)attention operations.
- ▶ Upsampling, normalization, activation, output convolution.

Binxu Wang, [Stable Diffusion: A Tutorial \(Harvard ML from Scratch\)](#).

- ▶ Meaning of a word depends on context, not always the same.
- ▶ Example: “I *book* a ticket to buy that *book*.”
- ▶ Word vectors should depend on context.
- ▶ Transformers let each word absorb influence from other words to become contextualized.

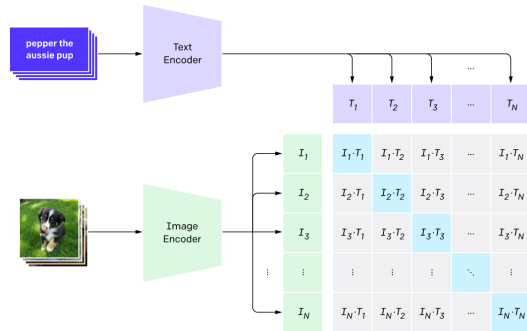


- ▶ Self-supervised learning of word representations.
- ▶ Predict missing / next words in a sentence (BERT, GPT).
- ▶ Contrastive learning: match image and text (CLIP).
- ▶ Downstream classifiers can decode part of speech, sentiment, ...



- ▶ Learn a joint encoding space for text captions and images.
- ▶ Maximize representation similarity between an image and its caption.
- ▶ Minimize similarity for other pairs.

1. Contrastive pre-training



Adapted from: Radford et al., [CLIP](#), [arXiv:2103.00020](#)

- ▶ Encoder in Stable Diffusion: pre-trained CLIP ViT-L/14 text encoder.
- ▶ Word vectors can also be randomly initialized and learned online.
- ▶ **Other conditional signals:**
 - Object categories (e.g. Shark, Trout): one vector per class.
 - Face attributes (e.g. {female, blonde hair, with glasses}): one vector per attribute.
- ▶ Time to be creative!

Pixel-space diffusion is redundant: the U-Net spends capacity on imperceptible high-frequency detail. **Stable Diffusion** (Rombach et al., 2022) decouples:

1. **Perceptual compression (VAE):** $\mathcal{E} : \mathbf{x} \mapsto \mathbf{z}$; remove imperceptible spatial detail, preserve layout.
2. **Semantic generation (LDM):** run diffusion on $\mathbf{z}$; lower memory, faster training.
3. **Decode:** $\mathcal{D}(\mathbf{z}) \mapsto \mathbf{x}$; project back to pixels.

Typical spatial downsampling: $8\times$ (e.g., $512 \times 512 \times 3$ pixels $\rightarrow 64 \times 64 \times 4$ latents, a $\sim 48\times$ reduction in dimensions).

Pipeline: CLIP text encoder $\rightarrow$ token embeddings $\mathbf{c} \in \mathbb{R}^{L \times d}$.

- ▶ **Self-attention:** Q, K, V all come from spatial latent features; provides global visual coherence.
- ▶ **Cross-attention:** Q from spatial features; K, V from the text embeddings; aligns image patches with prompt tokens.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

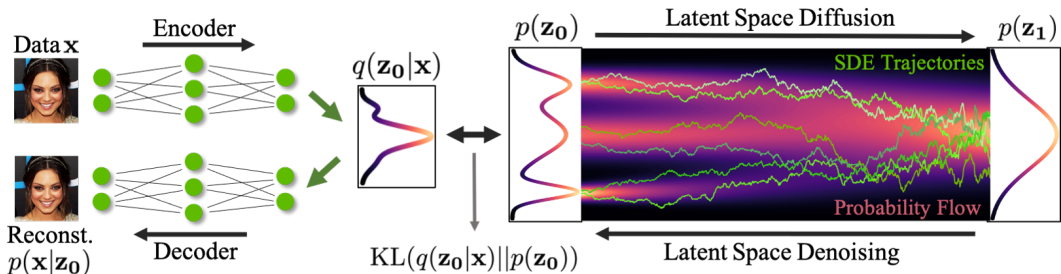
Scaling by $\sqrt{d_k}$ prevents softmax saturation in high dimensions.

Token length $L \leq 77$ (CLIP); multi-head attention throughout the U-Net blocks.

Stable Diffusion:

A Latent Diffusion Model (LDM)

What is a Latent Space?



- ▶ A **latent space** is an abstract, compressed space where high-dimensional data (like images) is represented in a lower-dimensional form.
- ▶ Think of it as a hidden layer of meaning—each point in this space represents a potential image or concept.
- ▶ In classical models like GANs or VAEs, the latent space is directly sampled from (e.g., a vector $\mathbf{z}$), and the generator turns it into an image.

What is a Latent Space? (cont.)

- ▶ **Latent** space = “Hidden space of ideas or features”

Two main ways latent spaces are used in diffusion:

► 1. Latent Diffusion Models (LDMs):

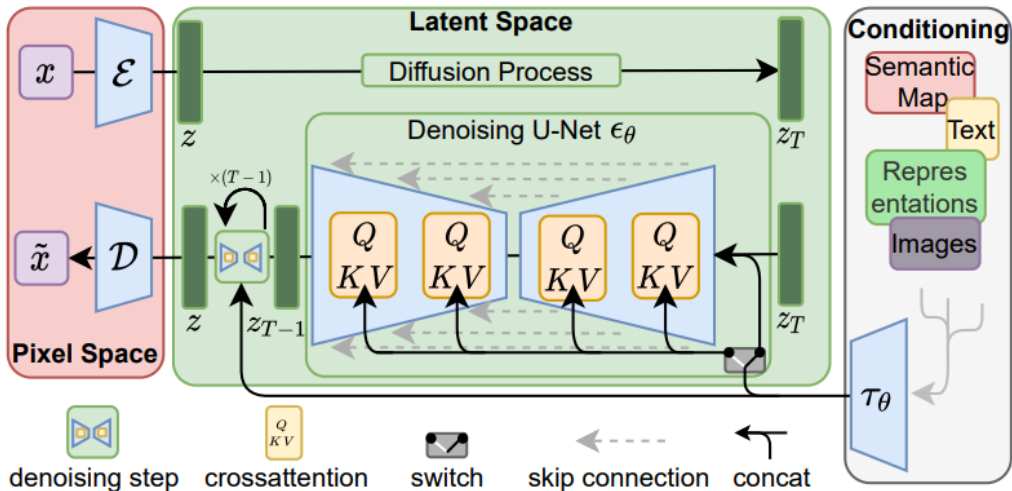
- Diffusion is performed in a compressed latent space, not directly on high-dimensional images.
- **Pipeline:** Compress image $\rightarrow$ Denoise in latent space $\rightarrow$ Decode back to image.
- Enables faster, cheaper training and inference, and allows for semantic control (e.g., text prompts, masks).

► 2. Latent Representations in Noise Vectors:

- Even in standard diffusion models, the initial noise vector acts as a latent code.
- Changing the noise vector generates different images or morphs between them.

- ▶ **Traditional pixel-space diffusion models are computationally expensive:** they operate directly on high-dimensional images, making both training and inference costly.
- ▶ **Latent diffusion reduces computational cost** by performing the diffusion process in a compressed, lower-dimensional latent space—while still maintaining high-quality image generation.
- ▶ This efficiency enables more complex and practical applications, such as text-guided generation and flexible image editing.
- ▶ **How is this achieved?** Instead of working in pixel space, we learn a compressed latent space using a Variational Autoencoder (VAE) with very low KL divergence weight (e.g., 10^{-6}). This yields a compact representation suitable for diffusion.
- ▶ This approach mirrors ideas from autoregressive models (e.g., VQGAN), enabling even more efficient and controllable generation pipelines.

Latent Diffusion Models: Visual Overview



Stable Diffusion:

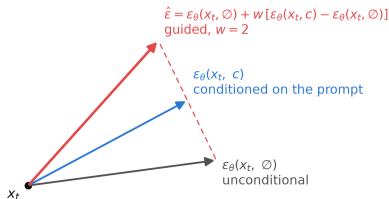
Guidance and Sampling

- ▶ Conditioning alone follows the prompt only loosely; every real system amplifies it with **classifier-free guidance** (Ho & Salimans, 2022).
- ▶ **Training trick:** randomly drop the text conditioning for a fraction of training steps ($\sim 10\%$), so one network learns both a **conditional** and an **unconditional** noise prediction.
- ▶ **At sampling time**, run both and **extrapolate**:

$$\hat{\epsilon} = \epsilon_{\theta}(x_t, \emptyset) + w [\epsilon_{\theta}(x_t, c) - \epsilon_{\theta}(x_t, \emptyset)]$$

- ▶ The guidance scale w trades **prompt fidelity** against **diversity**: $w = 1$ is plain conditional sampling; SD v1 defaults to $w \approx 7.5$; too high causes oversaturated, artifact-prone images.

Classifier-free guidance: extrapolate past the conditional prediction



- ▶ Each denoising step pushes the sample **past** the conditional prediction, away from the unconditional one: whatever the prompt adds gets amplified.
- ▶ **Negative prompts** reuse the same mechanism: replace the unconditional branch with an embedding of what you do *not* want, and the extrapolation pushes away from it.

- ▶ DDPM training uses ~ 1000 noise levels, but sampling does not have to visit all of them.
- ▶ **DDIM** (Song et al., 2021): a deterministic sampler over the same trained model; skips steps and enables consistent latent-space edits. Typically 20–50 steps.
- ▶ **ODE-solver view**: sampling is integrating a differential equation, so higher-order solvers (DPM-Solver++, Euler variants in practice) reach good quality in fewer steps.
- ▶ **Distillation** goes further, to 1–4 steps: latent consistency models (LCM), adversarial diffusion distillation (SDXL Turbo), and SD 3.5 **Large Turbo** all distill a many-step teacher into a few-step student.
- ▶ Practical takeaway: the model, the sampler, and the step count are **independent choices**; most quality-speed tuning happens here.

Stable Diffusion:

Customizing Stable Diffusion

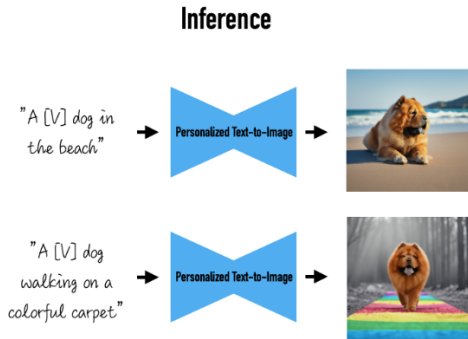
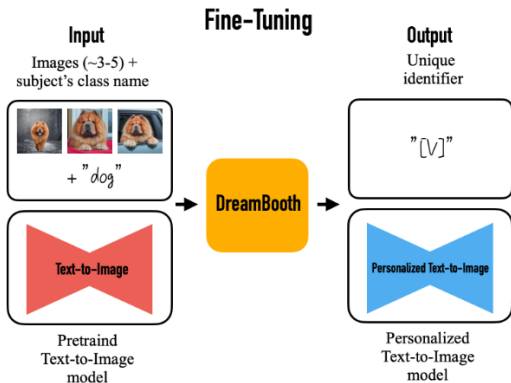
- ▶ Open weights made SD the platform people **fine-tune**; three techniques cover most of it:
- ▶ **Textual Inversion** (Gal et al., 2023): learn a **new token embedding** for a concept from a few images; the rest of the model stays frozen ($\sim$ kilobytes per concept).
- ▶ **DreamBooth** (Ruiz et al., 2023): fine-tune the model itself to bind a **rare token** to a specific subject (your pet, a product), with a prior-preservation loss against forgetting the class.
- ▶ **LoRA** (Hu et al., 2022): train **low-rank adapters** on the attention weights instead of the full model; megabytes instead of gigabytes, composable, and cheap to share. The de facto standard for style and character adaptation.

What is DreamBooth?

- ▶ A method for fine-tuning large diffusion models on a small number of images.
- ▶ Developed by Google Research (2022).
- ▶ Allows personalization of generative models with minimal data.

Key Concepts

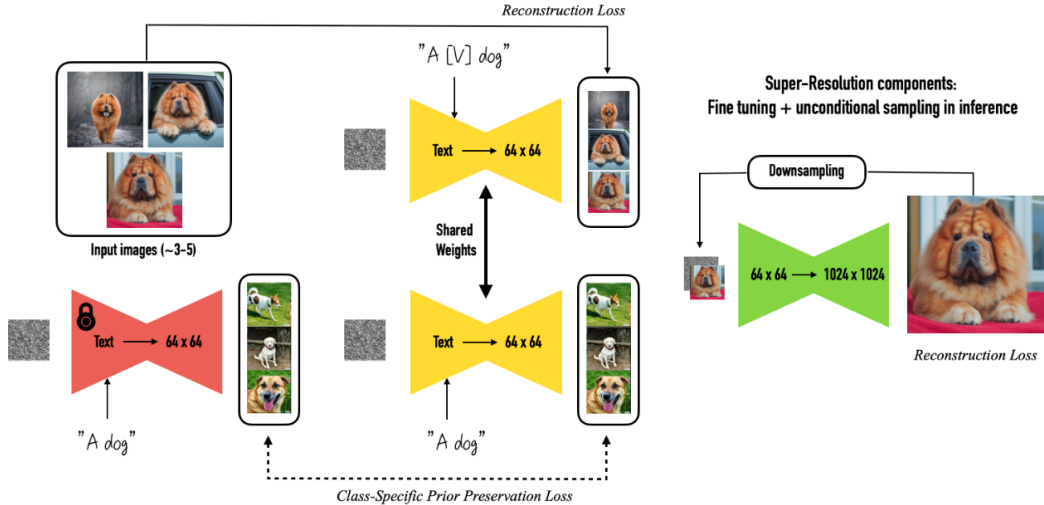
- ▶ **Personalization:** Adapts a pre-trained model to generate images of specific subjects (e.g., pets, people).
- ▶ **Fine-tuning:** Uses a small set of images to adjust the model's parameters.
- ▶ **Text Conditioning:** Leverages text prompts to guide image generation.



How Does DreamBooth Personalize Models?

- ▶ DreamBooth teaches a model to recognize a new subject using just a few photos.
- ▶ After training, the model can create new images of that subject in different places or situations.
- ▶ This is called “few-shot” learning—using only a few examples!

DreamBooth (cont.)



How Do We Tell the Model About the New Subject?

- ▶ We use a special made-up word (called a *rare-token*) to represent the new subject.
- ▶ For example: “a photo of `sks` dog at the beach”.
- ▶ The model learns to link this rare-token to the subject in your photos.

Why Use Rare-tokens?

- ▶ Rare-tokens are unique and not found in the model's original training data.
- ▶ This avoids mixing up your subject with things the model already knows.

How Does DreamBooth Avoid Forgetting?

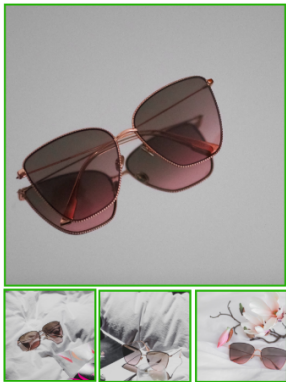
- ▶ If we only train on your subject, the model might forget how to make other images.
- ▶ To prevent this, we also show it regular class prompts (like “a photo of a dog”).
- ▶ We use a special loss to balance learning the new subject and keeping old knowledge:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{subject}} + \lambda \mathcal{L}_{\text{prior}}$$

where:

- $\mathcal{L}_{\text{subject}}$ = loss for your subject (rare-token prompt)
- $\mathcal{L}_{\text{prior}}$ = loss for general class (class prompt)
- λ = weight to balance the two

Input images



A [V] sunglasses in the jungle



A [V] sunglasses worn by a bear



A [V] sunglasses at Mt. Fuji



A [V] sunglasses on top of snow



A [V] sunglasses with Eiffel Tower in the background

[Ruiz et al., 2023](<https://arxiv.org/abs/2208.12242>)

Art Rendition

Original artistic renditions of our subject dog in the style of famous painters. We remark that many of the generated poses were not seen in the training set, such as the Van Gogh and Warhol rendition. We also note that some renditions seem to have novel composition and faithfully imitate the style of the painter - even suggesting some sort of creativity (extrapolation given previous knowledge).

Input images



Vincent Van Gogh



Michelangelo



Rembrandt



Johannes Vermeer



Pierre-Auguste Renoir



Leonardo da Vinci

[Ruiz et al.,

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**

Microsoft Corporation

{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu

LoRA Overview

Low-Rank Adaptation (LoRA) is a technique for efficient fine-tuning of large neural networks. Instead of updating all parameters, LoRA injects trainable low-rank matrices into each layer, significantly reducing the number of trainable parameters.

Key Idea: Decompose the weight update ΔW as a product of two low-rank matrices:

$$\Delta W = AB \tag{1}$$

where $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$, with $r \ll \min(d, k)$.

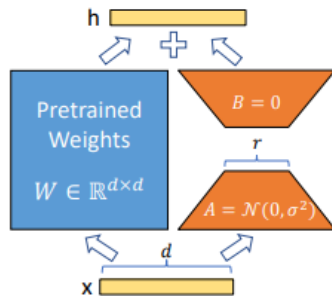


Figure 1: Our reparametrization. We only train A and B .

[Hu et al., 2021](<https://arxiv.org/abs/2106.09685>)

LoRA in Linear Layers

Consider a linear layer with weight $W_0 \in \mathbb{R}^{d \times k}$. In LoRA, the forward pass is modified as:

$$y = W_0 x + \alpha ABx \quad (2)$$

where α is a scaling factor, and only A and B are updated during fine-tuning.

Parameter Efficiency: The number of trainable parameters is reduced from $d \times k$ to $r \times (d + k)$.

LoRA Training Objective

The training objective remains the same as standard fine-tuning, e.g., minimizing a loss function $\mathcal{L}$:

$$\min_{A,B} \mathcal{L}(y, \hat{y}) \quad (3)$$

where y is the model output and $\hat{y}$ is the ground truth.

Regularization: Optionally, regularization terms can be added to encourage low-rank structure or prevent overfitting.

LoRA in Attention Mechanisms

LoRA is commonly applied to the query and value projection matrices in attention layers:

$$Q' = Q + \Delta Q = Q + A_q B_q \quad (4)$$

$$V' = V + \Delta V = V + A_v B_v \quad (5)$$

where A_q, B_q, A_v, B_v are low-rank matrices for the query and value projections, respectively.

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$ 0.1	94.2 $\pm$ 1.1	88.5 $\pm$ 1.1	60.8 $\pm$ 4.4	93.1 $\pm$ 1.1	90.2 $\pm$ 0.1	71.5 $\pm$ 2.7	89.7 $\pm$ 3.3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$ 1.1	94.7 $\pm$ 3.3	88.4 $\pm$ 1.1	62.6 $\pm$ 9.9	93.0 $\pm$ 2.2	90.6 $\pm$ 0.1	75.9 $\pm$ 2.2	90.3 $\pm$ 1.1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 3.3	95.1$\pm$2.2	89.7 $\pm$ 7.7	63.4 $\pm$ 1.2	93.3$\pm$3.3	90.8 $\pm$ 1.1	86.6$\pm$7.7	91.5$\pm$2.2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$2.2	96.2 $\pm$ 5.5	90.9$\pm$1.2	68.2$\pm$1.9	94.9$\pm$3.3	91.6 $\pm$ 1.1	87.4$\pm$2.5	92.6$\pm$2.2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$ 3.3	96.1 $\pm$ 3.3	90.2 $\pm$ 7.7	68.3$\pm$1.0	94.8$\pm$2.2	91.9$\pm$1.1	83.8 $\pm$ 2.9	92.1 $\pm$ 7.7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$3.3	96.6$\pm$2.2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8$\pm$3.3	91.7 $\pm$ 2.2	80.1 $\pm$ 2.9	91.9 $\pm$ 4.4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$ 5.5	96.2 $\pm$ 3.3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 2.2	92.1 $\pm$ 1.1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$ 3.3	96.3 $\pm$ 5.5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 2.2	91.5 $\pm$ 1.1	72.9 $\pm$ 2.9	91.5 $\pm$ 5.5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$2.2	96.2 $\pm$ 5.5	90.2$\pm$1.0	68.2 $\pm$ 1.9	94.8$\pm$3.3	91.6 $\pm$ 2.2	85.2$\pm$1.1	92.3$\pm$5.5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$2.2	96.9 $\pm$ 2.2	92.6$\pm$6.6	72.4$\pm$1.1	96.0$\pm$1.1	92.9$\pm$1.1	94.9$\pm$4.4	93.0$\pm$2.2	91.3

Table 2: RoBERTa_{base}, RoBERTa_{large}, and DeBERTa_{XXL} with different adaptation methods on the GLUE benchmark. We report the overall (matched and mismatched) accuracy for MNLI, Matthew’s correlation for CoLA, Pearson correlation for STS-B, and accuracy for other tasks. Higher is better for all metrics. * indicates numbers published in prior works. † indicates runs configured in a setup similar to Houlsby et al. (2019) for a fair comparison.

[Hu et al., 2021](<https://arxiv.org/abs/2106.09685>)

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$.1	8.85$\pm$.02	46.8$\pm$.2	71.8$\pm$.1	2.53$\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49$\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$.1	8.89$\pm$.02	46.8$\pm$.2	72.0$\pm$.2	2.47 $\pm$.02

Table 3: GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments we ran. * indicates numbers published in prior works.

- ▶ Text describes *what*; it is poor at *where*. **ControlNet** (Zhang et al., 2023) adds spatial conditioning: edge maps, human pose, depth, sketches, segmentation.
- ▶ Mechanism: clone the U-Net encoder into a **trainable copy** that ingests the condition image, keep the original weights **frozen**, and connect the copy through **zero-initialized convolutions** so training starts from a no-op and cannot damage the base model.
- ▶ Result: precise layout control with modest paired data, reusing everything the base model knows.
- ▶ Together with LoRA and inpainting, this is why an open 2022 model still anchors production pipelines today.

Stable Diffusion:

Applications and Lineage

- ▶ Initialize from Stable Diffusion.
- ▶ Add temporal layers (e.g., 1D Conv, temporal attention).
- ▶ Finetune on video data.
- ▶ Data pipeline:
 - Scrape **unlabeled** videos from the internet.
 - Use image and video captioners to generate synthetic text labels.
 - Apply aggressive data filtering using several heuristics (CLIP score, optical flow score, aesthetic scores, OCR, etc.).

Application: Stable Video Diffusion (cont.)





“a man wearing a white hat”

Image Inpainting with GLIDE

⁰GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models



a shiba inu wearing a beret and black turtleneck



a close up of a handpalm with leaves growing from it

Text-to-image generation with DALL·E 2

⁰ Hierarchical Text-Conditional Image Generation with CLIP Latents



Fix the CLIP embedding z_c
Decode using different decoder latents $\mathbf{x}_T$

Image Variations



Interpolate image CLIP embeddings z_i

Use different x_T to get different interpolation trajectories.

Image interpolation



a photo of a cat → an anime drawing of a super saiyan cat, artstation



a photo of a victorian house → a photo of a modern house



a photo of an adult lion → a photo of lion cub

Change the image CLIP embedding towards the difference of the text CLIP embeddings of two prompts.

Decoder latent is kept as a constant.

Text Difference Image interpolation



A brain riding a rocketship heading towards the moon.



A dragon fruit wearing karate belt in the snow.



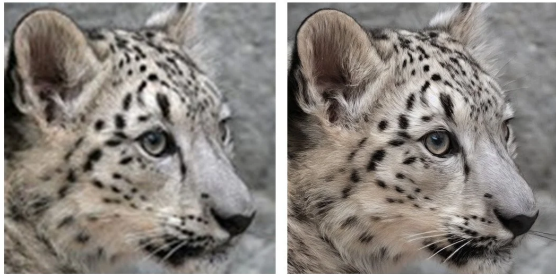
A relaxed garlic with a blindfold reading a newspaper while floating in a pool of tomato soup.

What changed vs. Stable Diffusion v1?

- ▶ **New text encoder:** OpenCLIP (LAION + Stability AI) replaces CLIP ViT-L/14—stronger language understanding and image quality.
- ▶ **Native resolutions:** text-to-image at **512×512** and **768×768**.
- ▶ **Upscaler diffusion:** ×4 super-resolution (e.g. 128→512).
- ▶ **depth2img:** depth-guided image-to-image—structure-preserving synthesis from inferred depth + text.
- ▶ **Updated inpainting:** text-guided inpainting fine-tuned on the Stable Diffusion 2.0 base model.

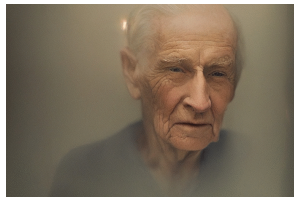
⁰Lopez, “Stable Diffusion 2.0 Release,” Stability AI (Nov 2022).

- ▶ Dedicated **upscaler diffusion** model enhances resolution by $\times 4$.
- ▶ Example: **128×128** low-res sample $\rightarrow$ **512×512** output.



Left: 128×128 . Right: 512×512 upscaled.

- ▶ **depth2img** infers depth from an input image, then generates new images conditioned on **text + depth**.
- ▶ Enables structure-preserving image-to-image and shape-conditional synthesis.
- ▶ Can produce radically different appearances while preserving spatial coherence and depth.



Input → variants (top); depth coherence preserved (bottom).



Text-guided inpainting model fine-tuned on the SD 2.0 text-to-image base.

⁰Stability AI, Stable Diffusion 2.0 Release.

- ▶ **SDXL** (Podell et al., 2023): the scaled-up successor to SD 2; for a long time the most-used open text-to-image model.
- ▶ A 2.6B-parameter U-Net (about $3\times$ the 860M of SD 1/2) with **two text encoders** used together (OpenCLIP ViT-bigG + CLIP ViT-L).
- ▶ Native **1024×1024** generation, plus micro-conditioning on image size and crop so the model learns from mixed-resolution data without artifacts.
- ▶ Optional **refiner** model: a second latent-diffusion stage specialized in high-frequency detail.

- ▶ SD 3 changes both the backbone and the training objective (Esser et al., 2024).
- ▶ **Backbone:** the U-Net is replaced by a **multimodal diffusion transformer (MM-DiT)**, following the DiT line (Peebles & Xie, 2023): text and image tokens flow through a joint transformer with separate weights per modality.
- ▶ **Objective: rectified flow**, i.e. flow matching on straight noise-data paths, instead of the DDPM objective; the paper's ablations found it superior at scale. (See the flow-matching section of the Normalizing Flows lecture.)
- ▶ **Text encoders:** two CLIP encoders plus **T5-XXL** for long, compositional prompts and better in-image text.
- ▶ **FLUX.1** (Black Forest Labs, 2024), built by SD's original authors, continues this rectified-flow-transformer lineage as the strongest open release of its generation.



Prompt: photo of three potions: the first potion is blue with the label "MANA", the second potion is red with the label "HEALTH", the third potion is green with the label "POISON". Old apothecary.

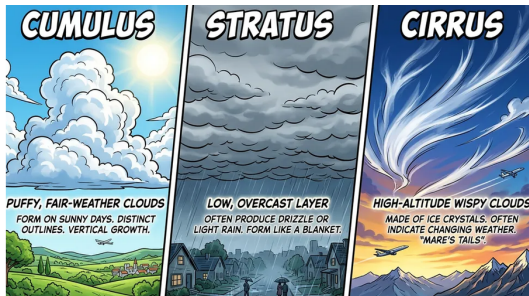
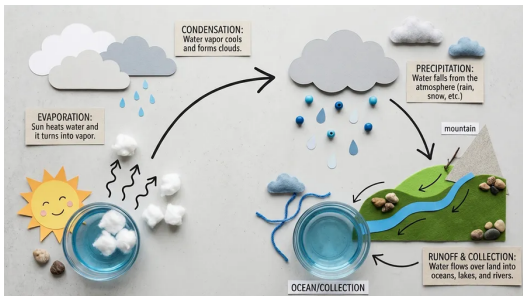


Prompt: scene of a giant ancient tortoise with a fantasy city built on its back. The tortoise's shell is covered in lush, dense forest with towering trees and a hidden, misty village nestled in the foliage. The city consists of intricately designed buildings that blend seamlessly with the natural environment, featuring rope bridges connecting different sections of the city.



Prompt: collage art 'We're Leaving For the Future' 1980s #vaporwave aesthetic internet art glued layered magazine cutout image shape scrap, torn ragged paper art, BASIC code, halftone, #pixelart.

- **Customizability:** Large (8.1B), Large Turbo (8.1B) and Medium (2.5B) models.
- **Efficient Performance:** Optimized to run on standard consumer hardware; Medium and Large Turbo require no specialized GPUs.
- **Highly Accessible:** Medium needs only 9.9 GB of VRAM, making advanced diffusion imaging feasible on most consumer GPUs.



Examples: water-cycle infographic and cloud-type comparison.

- Builds on Google's Gemini Image line: Nano Banana (2025) → Nano Banana Pro → Nano Banana 2.
- Production-ready output: multiple aspect ratios and resolutions from 512px to 4K.

⁰Raisinghani, "Nano Banana 2: Combining Pro capabilities with lightning-fast speed," Google DeepMind blog (Feb 2026).

Text to Image generation with stable diffusion.
<https://huggingface.co/spaces/stabilityai/stable-diffusion>

Stable Diffusion:

Summary & Limitations

From diffusion to Stable Diffusion

- ▶ Diffusion models learn to reverse a gradual noising process; sampling remains iterative but training is stable and scalable.
- ▶ **Latent Diffusion (LDM):** denoise in a VAE-compressed latent space ($\sim 8\times$ downsampling) rather than full pixel space—lower memory and faster training/inference.

Core architecture

- ▶ **Attention U-Net:** encoder–decoder with skip connections; downsampling / upsampling for multiscale features.
- ▶ **Time conditioning:** Fourier-feature time embeddings added per channel.
- ▶ **Text conditioning:** CLIP (ViT-L/14 in SD v1) maps prompts to token embeddings; cross-attention injects language into the U-Net.

Using and steering the model

- ▶ **Classifier-free guidance:** extrapolate past the conditional prediction with scale w ; negative prompts reuse the same mechanism.
- ▶ **Samplers:** DDIM and higher-order ODE solvers cut 1000 steps to 20–50; distillation (LCM, Turbo) reaches 1–4.
- ▶ **Customization:** Textual Inversion, DreamBooth, and LoRA personalize; ControlNet adds spatial control.

Model evolution & applications

- ▶ **SD 2.0:** OpenCLIP text encoder; native 512/768 T2I; upscaler ($\times 4$), depth2img, and updated inpainting.
- ▶ **SDXL:** 3.5B U-Net, dual text encoders, native 1024², optional refiner.
- ▶ **SD 3 / 3.5:** MM-DiT transformer backbone trained with rectified flow; Large 8.1B, Large Turbo, Medium 2.5B run on consumer GPUs; FLUX.1 continues the lineage.
- ▶ Broader ecosystem: video (Stable Video Diffusion), inpainting/editing (GLIDE, DALL·E 2), and newer multimodal systems (e.g. Nano Banana 2).

- ▶ **Slow sampling** — many denoising steps (partially addressed by distilled models such as SD 3.5 Large Turbo).
- ▶ **Data & compute** — high-quality open models still require large-scale curated data and substantial GPU resources to train.
- ▶ **Prompt sensitivity** — output quality and adherence depend on prompt wording; base models may vary across seeds.
- ▶ **Distribution gaps** — rare concepts, fine details, and compositional reasoning remain challenging.
- ▶ **Safety & misuse** — text-to-image systems raise concerns around deepfakes, copyright, and misinformation.

Tutorials and Surveys

- ▶ Binxu Wang: [Stable Diffusion: A Tutorial \(Harvard ML from Scratch\)](#) — U-Net, time embedding, CLIP conditioning.
- ▶ Kreis, K., Gao, R., & Vahdat, A.: [CVPR 2022 Tutorial: Diffusion Models](#)
- ▶ Rogge, N., & Rasul, K.: [The Annotated Diffusion Model](#)
- ▶ Lilian Weng: [What are Diffusion Models?](#)
- ▶ Yang Song: [Score-Based Generative Modeling: A Brief Overview](#)

Foundational Diffusion Models

- ▶ Ho, J., Jain, A., & Abbeel, P. (2020). [Denoising Diffusion Probabilistic Models](#)
- ▶ Sohl-Dickstein, J., et al. (2015). [Deep Unsupervised Learning using Nonequilibrium Thermodynamics](#)
- ▶ Song, Y., & Ermon, S. (2019). [Generative Modeling by Estimating Gradients of the Data Distribution](#)
- ▶ Nichol, A., & Dhariwal, P. (2021). [Improved Denoising Diffusion Probabilistic Models](#)

Stable Diffusion: Core Papers

- ▶ Rombach, R., et al. (2022). **High-Resolution Image Synthesis with Latent Diffusion Models** — Stable Diffusion / LDM (VAE + U-Net + conditioning).
- ▶ Radford, A., et al. (2021). **Learning Transferable Visual Models From Natural Language Supervision (CLIP)** — text encoder and cross-attention conditioning.

Guidance, Sampling, and Customization

- ▶ Ho, J., & Salimans, T. (2022). [Classifier-Free Diffusion Guidance](#)
- ▶ Song, J., Meng, C., & Ermon, S. (2021). [Denoising Diffusion Implicit Models \(DDIM\)](#), ICLR 2021.
- ▶ Lu, C., et al. (2022). [DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models](#)
- ▶ Luo, S., et al. (2023). [Latent Consistency Models](#)
- ▶ Sauer, A., et al. (2023). [Adversarial Diffusion Distillation \(SDXL Turbo\)](#)
- ▶ Gal, R., et al. (2023). [An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion](#), ICLR 2023.
- ▶ Ruiz, N., et al. (2023). [DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation](#), CVPR 2023.
- ▶ Hu, E., et al. (2022). [LoRA: Low-Rank Adaptation of Large Language Models](#), ICLR 2022.

- ▶ Zhang, L., Rao, A., & Agrawala, M. (2023). Adding Conditional Control to Text-to-Image Diffusion Models (ControlNet), ICCV 2023.

Applications and Model Releases

- ▶ Nichol, A., et al. (2021). [GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models](#)
- ▶ Ramesh, A., et al. (2022). [Hierarchical Text-Conditional Image Generation with CLIP Latents \(DALL·E 2\)](#)
- ▶ Saharia, C., et al. (2022). [Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding \(Imagen\)](#)
- ▶ Blattmann, A., et al. (2023). [Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets](#)
- ▶ Podell, D., et al. (2023). [SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis](#), ICLR 2024.
- ▶ Peebles, W., & Xie, S. (2023). [Scalable Diffusion Models with Transformers \(DiT\)](#), ICCV 2023.

- ▶ Esser, P., et al. (2024). [Scaling Rectified Flow Transformers for High-Resolution Image Synthesis \(Stable Diffusion 3\)](#), ICML 2024.
- ▶ Black Forest Labs (2024). [Announcing Black Forest Labs \(FLUX.1\)](#)
- ▶ Lopez, J. (2022). [Stable Diffusion 2.0 Release](#) — OpenCLIP, upscaler, depth2img, inpainting (Stability AI blog).
- ▶ Stability AI (2024). [Introducing Stable Diffusion 3.5](#) — SD 3.5 Large / Large Turbo / Medium (Stability AI blog).
- ▶ Raisinghani, A. (2026). [Nano Banana 2: Combining Pro capabilities with lightning-fast speed](#) (Google DeepMind blog).